

Programação Funcional

Aula 03 - Listas e tuplas em Haskell

Eliseu C. Miguel

Universidade Federal de Alfenas

11 de agosto de 2025

Tópicos

- 1 Introdução às Listas
- 2 Operadores e Funções Básicas
- 3 Funções Recursivas sobre Listas
- 4 Exemplo de Algoritmo
- 5 Introdução às Tuplas
- 6 Exercícios

Tópicos

- 1 Introdução às Listas
- 2 Operadores e Funções Básicas
- 3 Funções Recursivas sobre Listas
- 4 Exemplo de Algoritmo
- 5 Introdução às Tuplas
- 6 Exercícios

Tópicos

- 1 Introdução às Listas
- 2 Operadores e Funções Básicas
- 3 Funções Recursivas sobre Listas
- 4 Exemplo de Algoritmo
- 5 Introdução às Tuplas
- 6 Exercícios

Tópicos

- 1 Introdução às Listas
- 2 Operadores e Funções Básicas
- 3 Funções Recursivas sobre Listas
- 4 Exemplo de Algoritmo
- 5 Introdução às Tuplas
- 6 Exercícios

Tópicos

- 1 Introdução às Listas
- 2 Operadores e Funções Básicas
- 3 Funções Recursivas sobre Listas
- 4 Exemplo de Algoritmo
- 5 Introdução às Tuplas
- 6 Exercícios

Tópicos

- 1 Introdução às Listas
- 2 Operadores e Funções Básicas
- 3 Funções Recursivas sobre Listas
- 4 Exemplo de Algoritmo
- 5 Introdução às Tuplas
- 6 Exercícios

Tópicos

- 1 Introdução às Listas
- 2 Operadores e Funções Básicas
- 3 Funções Recursivas sobre Listas
- 4 Exemplo de Algoritmo
- 5 Introdução às Tuplas
- 6 Exercícios

Listas em Haskell

- Coleções ordenadas de elementos do mesmo tipo
- Tipo denotado por `[t]` onde `t` é o tipo dos elementos

Exemplos

```
[1,2,3] :: [Int]
[True,False] :: [Bool]
['a','b','c'] :: [Char]
"abc" :: [Char]
[] :: [t] (lista vazia)
```

Listas complexas

```
[[1,2],[3]] :: [[Int]]
[sum,product] :: [Int -> Int]
```

Casamento de padrão

- Podemos casar padrão com listas com `[]` (lista vazia)
- Além disso, podemos casar padrão com `(a:x)`, onde `a` é a cabeça da lista e `x` é cauda;

Funções `head'` e `tail'`

```
head' (a:x) = a
```

```
head' [1,2,3,4] = 1
```

```
tail' (a:x) = x
```

```
tail' [1,2,3,4] = [2,3,4]
```

Construção de Listas

- Ordem e repetição importam:

Listas distintas

$[1,2] \neq [2,1]$

$[1,1,2] \neq [1,2]$

- Construção com intervalos:

Ranges

$[1..5] = [1,2,3,4,5]$

$[1,3..9] = [1,3,5,7,9]$

$[10,8..0] = [10,8,6,4,2,0]$

Operador Cons (:)

- Adiciona elemento no início da lista
- Tipo: $t \rightarrow [t] \rightarrow [t]$

Exemplos

$1:[2,3] = [1,2,3]$

$'a':"bc" = "abc"$

$[]:[1,2] = [[], [1,2]]$

Erro comum

$1:2:3:[] \checkmark$

$[]:1:2 \times$

Operador de Concatenação (++)

- Concatena duas listas do mesmo tipo
- Tipo: $[t] \rightarrow [t] \rightarrow [t]$

Exemplos

$[1,2]++[3] = [1,2,3]$

$"a"++"b" = "ab"$

Diferença entre $:$ e $++$

$1:[2,3] \neq [1]++[2,3]$

(mesmo resultado, mas operações diferentes)

Função sumList

Definição

```
sumList :: [Int] -> Int  
sumList [] = 0  
sumList (x:xs) = x + sumList xs
```

Exemplo de execução

```
sumList [1,3,2]  
= 1 + sumList [3,2]  
= 1 + 3 + sumList [2]  
= 1 + 3 + 2 + sumList []  
= 1 + 3 + 2 + 0 = 6
```

Função double

Definição

```
double :: [Int] -> [Int]
double [] = []
double (x:xs) = (2*x) : double xs
```

Exemplo de execução

```
double [5,3]
= (2*5) : double [3]
= 10 : (6 : double [])
= 10 : (6 : []) = [10,6]
```

Busca em Lista

Definição

```
membro :: Int -> [Int] -> Bool  
membro _ [] = False  
membro x (y:ys) | x == y = True  
                 | otherwise = membro x ys
```

Exemplo

```
membro 3 [5,3,2] = True  
membro 4 [5,3,2] = False
```


Tuplas em Haskell

- Coleções ordenadas de elementos de tipos possivelmente diferentes
- Tamanho fixo e definido pelo número de componentes
- Tipo denotado por $(t_1, t_2, \dots, t_n)$

Exemplos

```
(1, 'a') :: (Int, Char)
("Joao", 25) :: (String, Int)
(True, "a", 3.0) :: (Bool, String, Float)
```

Funções fst e snd

```
fst (1, 'a') = 1
snd (1, 'a') = 'a'
(Apenas para tuplas de 2 elementos)
```

Casamento de Padrão em Tuplas

Definição

```
somaPar :: (Int, Int) -> Int  
somaPar (x,y) = x + y
```

Extraindo múltiplos valores

```
dados = ("Alice", 30)  
nome (n, _) = n  
idade (_, i) = i  
nome dados = "Alice"  
idade dados = 30
```

Exercícios

Utilizando as funções abaixo, realize os exercícios propostos

Funções auxiliares

```
perodo::Int  
perodo = 7  
maxi :: Int -> Int -> Int  
maxi m n  
    | m >= n = m  
    | otherwise = n
```

Exercícios

Tabela de vendas

`vendas :: Int -> Int`

`vendas 0 = 0`

`vendas 1 = 41`

`vendas 2 = 72`

`vendas 3 = 48`

`vendas 4 = 0`

`vendas 5 = 91`

`vendas 6 = 55`

`vendas 7 = 30`

Exercícios

- 1 Escreva uma função que retorna uma lista de vendas
- 2 Escreva uma função que retorna `[[Int]]` com listas de dia e venda
- 3 Escreva uma função que ordena uma lista de inteiros
- 4 Escreva uma função que gera uma lista de tuplas com dia e venda
- 5 Escreva uma função que retorna o dia da maior venda

Referências Bibliográficas

Book [1]



Vladimir Oliveira Di Iorio.

Programação Funcional - Tipos Básicos e Programas Simples em Haskell.

DPI - UFV.

Sobre as apresentações didáticas

Atenção: este material não é completo para seus estudos.
Ao contrário, é apenas um guia para lhe informar quais são os
tópicos importantes a serem estudados.

O estudo completo e necessário dá-se por materiais específicos
sobre cada tópico, como livros e conteúdo na Internet.

Selecione materiais de fontes de seu interesse considerando a
bibliografia citada no plano de ensino da disciplina para estudar os
tópicos aqui abordados.

Consulte o professor, em caso de dúvidas

Agradecimentos especiais

Agradeço ao monitor da disciplina Felipe Araújo Correia por ter elaborado esta apresentação.

Agradeço a toda a comunidade L^AT_EX.
Em especial a *Till Tantau* pelo *Beamer*.

<https://www.tcs.uni-luebeck.de/mitarbeiter/tantau/>

Desta forma, tornou-se possível a escrita deste material didático.